Web API Routing

Overview



Routing in ASP.Net Core Web API

Types of Routing

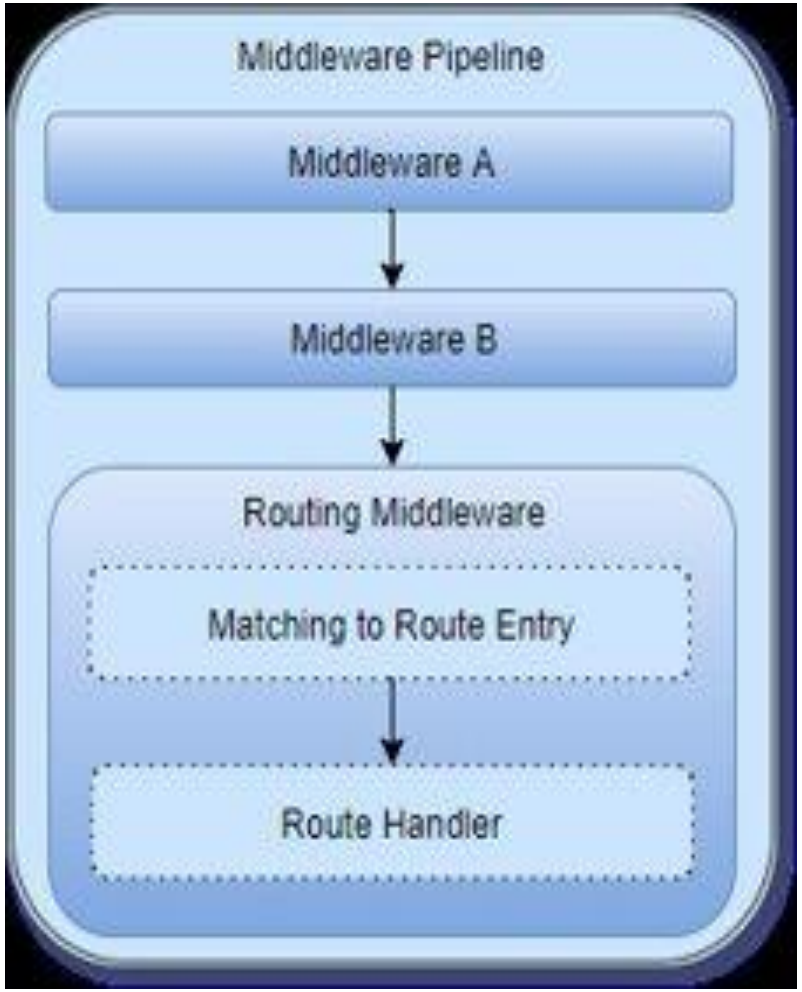
Routing Constraints

Routing in ASP.Net Core Web API

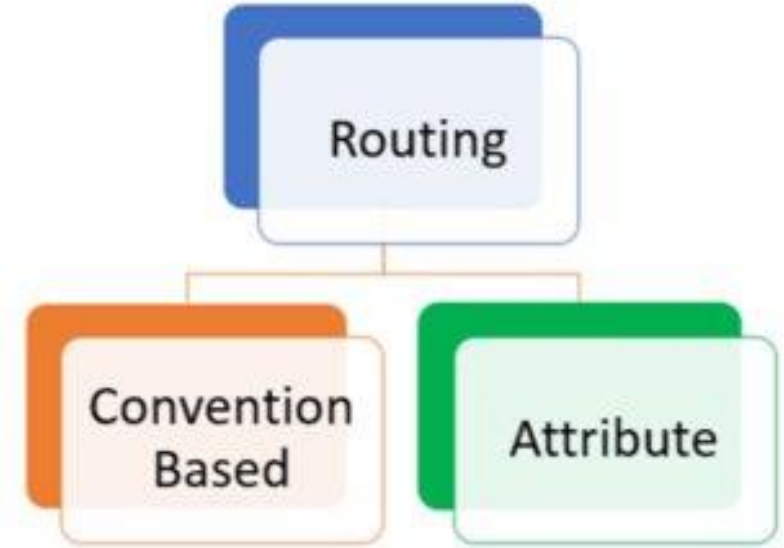
What is Routing?

- Routing is responsible for matching incoming HTTP requests and dispatching those requests to the app's executable endpoints.
- Endpoints are the app's units of executable request-handling code. Endpoints are defined in the app and configured when the app starts.
- The endpoint matching process can extract values from the request's URL and provide those values for request processing.
- Using endpoint information from the app, routing is also able to generate URLs that map to endpoints.

Routing Middleware



Types of Routing

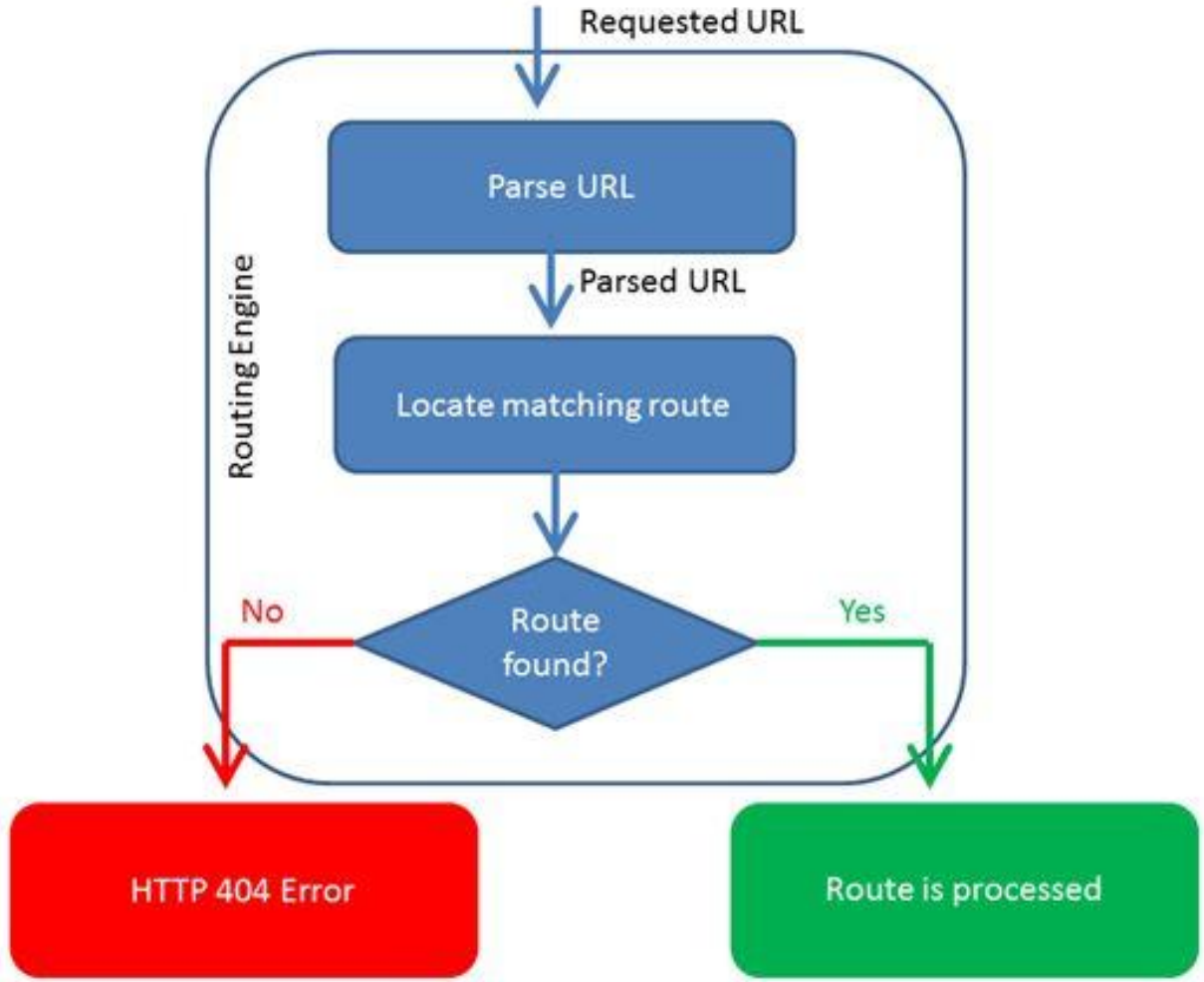


- **Conventional routing**

Once a fresh ASP.net Core MVC application is created by making use of the default template, the app configures a default routing.

- **Attribute routing**

It will be possible to use the Attribute Routing feature by placing a route on the action method or the controller.


Requested URL
Parse URL
Parsed URL
Routing Engine
Locate matching route
No
Route found?
Yes
HTTP 404 Error
Route is processed

Convention Based Routing

By default, ASP.NET Core MVC sets up convention-based routing when you create a new project. The conventions follow the traditional controller and action naming patterns. Here's how it works:

- **Controller Name Convention:** By convention, the framework assumes that controllers end with "Controller" in their class names. For example, a controller for handling products might be named ProductController.
- **Action Name Convention:** For each controller, the framework looks for public methods that return IActionResult (or its derived types) and considers them as actions. The naming convention for actions is to use the action name in lowercase. For example, an action for displaying product details might be named Details.
- **URL Pattern:** The URL pattern for convention-based routing follows this format: */Controller/Action/{id?}*. The {id?} part denotes an optional parameter named id.

Here's a simple example of how convention-based routing works:

In the Program.cs file, By default, it sets up convention-based routing:

```
// Other configurations...

    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapControllerRoute(
            name: "default",
            pattern: "{controller=Home}/{action=Index}/{id?}");
    });
    app.Run();
```

The MapControllerRoute method in Program.cs sets up the default convention-based route with optional id parameter. However, you can create more complex routes by customizing the MapControllerRoute method with additional parameters as needed.

```
[ApiController]
public class ProductsController : ControllerBase
{
    [HttpGet]
    public ActionResult<IEnumerable<string>> Get()
    {
        // Get list of products
        // ...
    }

    [HttpGet]
    public ActionResult<string> Get(int id)
    {
        // Get a specific product by ID
        // ...
    }
    [HttpPost]
    public ActionResult<string> Post([FromBody] Product product)
    {
        // Create a new product
        // ...
    }
```

```
    [HttpPut]
    public ActionResult<string> Put(int id, [FromBody]
Product product)
    {
        // Update an existing product
        // ...
    }

    [HttpDelete)]
    public ActionResult<string> Delete(int id)
    {
        // Delete a product by ID
        // ...
    }
}
```

Attribute Based Routing

To use attribute-based routing, you need to enable it in your ASP.NET Core MVC application. By default, both convention-based and attribute-based routing are enabled, so you can use either or both approaches as needed.

Here's how to use attribute-based routing in ASP.NET Core MVC:

- **Enable Attribute Routing:**

In the Program.cs file, ensure that the attribute routing is enabled by adding app.UseEndpoints and removing the convention-based routing setup:

```
// Other configurations...

    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapControllers(); // Enable attribute-based routing
    });

app.Run();
```

- **Define Routes with Attributes:**

GET

```
//localhost:[port]/api/Employee
[Route("api/[controller]")]
[ApiController]
public class EmployeeController : ControllerBase
{
        static List<EmpProfile> empProfiles = new List<EmpProfile>
        {
                new EmpProfile{EmpCode=100,EmpName="Scott",Email="scott@gmail.com"},
                new EmpProfile{EmpCode=101,EmpName="Tiger",Email="tiger@gmail.com"},
                new EmpProfile{EmpCode=102,EmpName="Smith",Email="smith@gmail.com"}
        };
                [HttpGet]
                [Produces("application/json")]
                [Route("GetAll")]
                public IActionResult GetAllEmployees()
                {
                        if (empProfiles != null)
                        {
                                return Ok( empProfiles);
                        }
                        else
                        {
                                return NotFound();
                        }
}
```

GET By id

```
[HttpGet]
[Produces("application/json")]
[Route("GetByCode/{id}")]
public IActionResult GetEmpByCode(int id)
{
        var empProfile = empProfiles.Where(emp=>emp.EmpCode==id).FirstOrDefault();
        if (empProfile != null)
        {
                return Ok(empProfile);
        }
        else
        {
                return NotFound();
        }
}
```

POST

```
[HttpPost]
[Route("AddEmp")]
public IActionResult PostEmp([FromBody]EmpProfile empProfile)
{
        if (empProfile != null)
        {
                empProfiles.Add(empProfile);
                return Ok();
        }
        else
        {
                return BadRequest();
        }
}
```

PUT

```
[HttpPut]
[Produces("application/json")]
[Route("UpdateEmp/{id}")]
public IActionResult PutEmployee(int id, [FromBody] EmpProfile empProfile)
{
        var ExistEmpProfile=empProfiles.Where(emp=>emp.EmpCode == id).FirstOrDefault();
        if (ExistEmpProfile != null)
        {
                ExistEmpProfile.EmpCode = empProfile.EmpCode;
                ExistEmpProfile.EmpName= empProfile.EmpName;
                ExistEmpProfile.Email= empProfile.Email;
                return Accepted();
        }
        else
        {
                return NotFound();
        }
}
```

DELETE

```
[HttpDelete]
[Route("DeleteEmp/{id}")]
public IActionResult DeleteEmp(int id)
{
        var EmpProfile = empProfiles.Where(emp => emp.EmpCode == id).FirstOrDefault();
        if(EmpProfile != null)
        {
                empProfiles.Remove(EmpProfile);
                return Ok();
                }
                else
                {
                return NotFound();
                }
        }
}
```

Routing Constraints

Common Built-in Constraints

Constraint	Description	Example
int	Only integers	{id:int}
long	Only long values	{id:long}
bool	true/false	{flag:bool}
datetime	Valid date	{date:datetime}
guid	Valid GUID	{id:guid}
alpha	Only alphabets	{name:alpha}
minlength(x)	Min length	{name:minlength(5)}
maxlength(x)	Max length	{name:maxlength(10)}
length(x,y)	Range length	{name:length(5,10)}
range(x,y)	Numeric range	{age:range(18,60)}
regex	Pattern matching	{code:regex(^[A-Z]{3}$)}

Why Use Constraints?

- Avoid invalid API calls
- Improve routing performance
- Prevent ambiguous routes
- Better API design (clean & predictable)

References

https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/routing

https://dotnettutorials.net/lesson/routing-in-asp-net-core-web-api/

https://www.c-sharpcorner.com/article/routing-in-restful-apis-using-net-core/

https://www.tutorialsteacher.com/webapi/web-api-routing

https://www.dotnettricks.com/learn/aspnetcore/routing

https://medium.com/quick-code/routing-in-asp-net-core-c433bff3f1a4

https://www.infoworld.com/article/3569369/how-to-use-attribute-routing-in-aspnet-core.html